

Manual de JMeter

¿Qué hacer para realizar un Plan de Pruebas?

Para realizar un Plan de Pruebas básico en JMeter, se deben realizar los siguientes pasos:

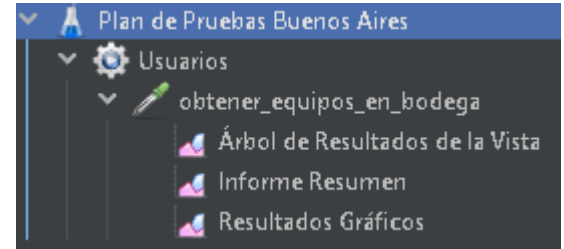
1. Iniciar JMeter
2. Crear un Plan de Pruebas (Test Plan)
3. Crear un Grupo de Usuarios Virtuales (Thread Group)
4. Agregar Peticiones o Solicitudes que Simulan las Acciones de los Usuarios (por ejemplo: Sampler tipo Http)
5. Agregar Recolectores de Datos o Analizadores de Resultados (Listeners)

Una vez definido el plan de pruebas, se verá más o menos como la imagen a la derecha.

Par efectos de explicar este manual, se usará la función:

"obtenerEquipos_en_bodega"

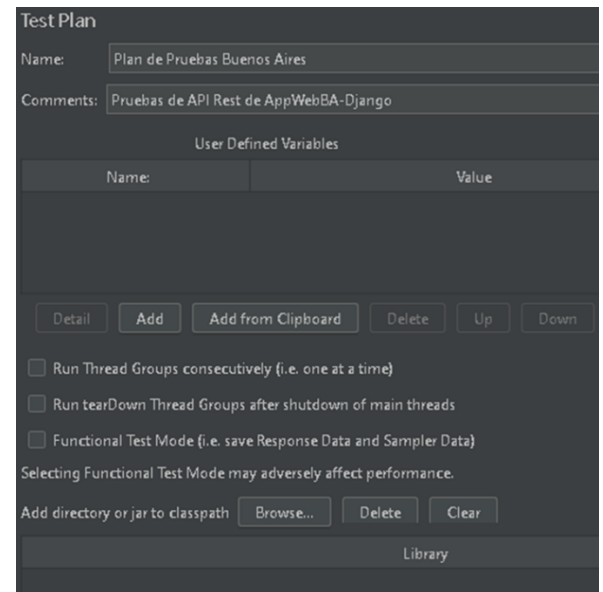
Esta función pertenece a las API REST de la aplicación Django AppWebBA del proyecto BuenosAires.



Crear un Plan de Pruebas (Test Plan)

Permite configurar el nombre y descripción del plan de pruebas, definir variables personalizadas, controlar el orden de ejecución de los grupos de hilos, habilitar el modo de prueba funcional y agregar bibliotecas externas al classpath.

Name	Es el nombre que se le asigna al plan de pruebas, en este caso.
Comments	Permite agregar comentarios o descripciones sobre el plan de pruebas.
User Defined Variables	Variables Definidas por el Usuario: permite definir variables personalizadas que se pueden utilizar en diferentes partes del plan de pruebas. Las variables pueden contener valores como URLs, parámetros de entrada, nombres de usuario, contraseñas, etc.
Run Thread Groups consecutively (i.e. one at a time):	Si está marcada, indica que los grupos de hilos dentro del plan de pruebas se ejecutarán de forma secuencial, sino se ejecutarán simultáneamente.
Run tearDown Thread Groups after shutdown of main threads	Si está marcada, indica que los grupos de hilos de "tearDown" (utilizados para realizar acciones de limpieza o cierre después de la prueba) se ejecutarán después de que los hilos principales hayan finalizado.
Functional Test Mode (i.e. save Response Data and Sampler Data):	Si está marcada, habilita el modo de prueba funcional, lo que significa que JMeter guardará datos adicionales sobre las respuestas y muestras (samplers). Esto puede ser útil para depurar errores, pero puede afectar el rendimiento de la prueba, especialmente en pruebas de carga con muchos usuarios virtuales.
Add directory or jar to classpath	Añadir directorio o jar al classpath: permite agregar directorios o archivos JAR (Java Archive) al classpath de JMeter. Esto puede ser necesario si se utilizan bibliotecas externas o plugins personalizados en el plan de pruebas.



Grupo un Grupo de Usuarios Virtuales (Thread Group) Usuarios

Es el punto de partida de cualquier plan de pruebas. Representa un grupo de usuarios virtuales que ejecutarán el mismo escenario de prueba, simulando el comportamiento de usuarios reales interactuando con tu aplicación o servicio web.

Cada hilo dentro del grupo de hilos actúa como un usuario individual, enviando solicitudes al servidor y recibiendo respuestas. Puedes configurar el número de hilos en el grupo, el tiempo que tardan en iniciarse (ramp-up period), el número de veces que repiten las solicitudes (loop count) y otras propiedades para simular diferentes escenarios de carga y medir el rendimiento de tu sistema.

¿Para qué sirve un grupo de hilos?

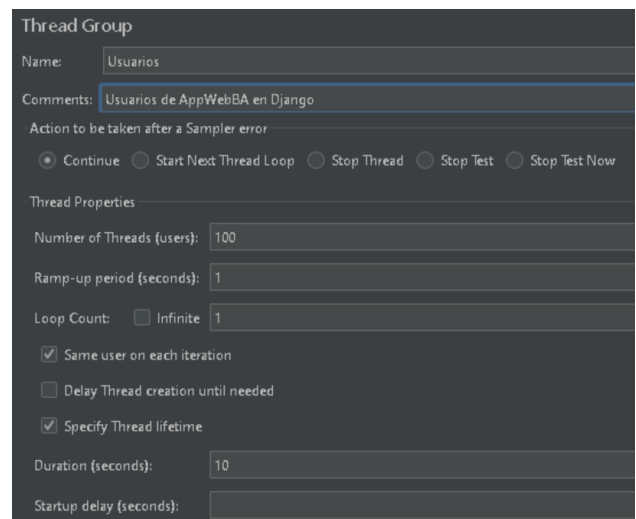
- **Simular usuarios:** Cada hilo representa un usuario virtual que interactúa con tu aplicación simulando tráfico.
- **Generar carga:** A mayor número de hilos, mayor es la carga, permitiendo evaluar el rendimiento bajo diferentes niveles de estrés.
- **Medir el rendimiento:** JMeter recopila datos sobre el tiempo de respuesta, la tasa de solicitudes por segundo y otros indicadores de rendimiento, permitiéndote identificar cuellos de botella y áreas de mejora en tu aplicación.
- **Probar diferentes escenarios:** Puedes configurar diferentes grupos de hilos con distintas propiedades para simular diferentes escenarios de uso, como usuarios concurrentes, picos de tráfico o patrones de uso específicos.

¿Cómo funciona un grupo de hilos?

1. **Configuración.** Define propiedades del grupo de hilos, como su número, el tiempo de ramp-up y el número de iteraciones.
2. **Inicio.** JMeter inicia los hilos de forma secuencial o simultánea según la configuración.
3. **Ejecución.** Cada hilo ejecuta solicitudes definidas dentro del grupo de hilos, como peticiones HTTP, llamadas a servicios web, etc.
4. **Recopilación de datos.** JMeter recopila datos del rendimiento de las solicitudes (tiempo de respuesta, código de estado, etc.)
5. **Finalización.** Cuando los hilos han completado sus iteraciones, JMeter finaliza la prueba y genera un informe con los resultados.

Para agregar un Thread Group se debe hacer clic derecho sobre el Plan de Pruebas y seleccionar Add → Threads (users) → ThreadGroup. Las siguientes opciones te permiten controlar el número de usuarios virtuales, la velocidad a la que se inician, el número de veces que repiten las solicitudes y cómo JMeter maneja los errores. Esto te ayuda a simular diferentes escenarios de carga y medir el rendimiento de tu aplicación bajo diversas condiciones.

Name	Descripción del grupo de hilos, útil para organizar tus pruebas.
Comments	Espacio para agregar notas o comentarios sobre el grupo de hilos.
Action to be taken after a Sampler error	Define qué hacer si una solicitud falla. Continue: Ignorar el error y seguir con las siguientes solicitudes, Start Next Thread Loop: Detener el hilo actual y comenzar la siguiente iteración, Stop Thread: Detener el hilo actual, Stop Test: Detener toda la prueba después de que los hilos actuales terminen y Stop Test Now: Detener inmediatamente toda la prueba.
Number of Threads (users)	Número de usuarios virtuales que simulará JMeter.
Ramp-up period (seconds)	Segundos que tardará JMeter en iniciar todos los hilos. Ej: si tienes 10 hilos y un tiempo de ramp-up de 10 segundos, se iniciará un hilo por segundo.
Loop Count	Número de veces que cada hilo ejecutará las solicitudes dentro del grupo de hilos. Si marcas "Infinite", los hilos continuarán ejecutando las solicitudes hasta que detengas la prueba manualmente.
Same user on each iteration	Si está marcado, cada hilo utilizará las mismas credenciales de usuario (si las hay) en todas las iteraciones. Si no está marcado, las credenciales se pueden variar.
Delay Thread creation until needed	Si está marcado, JMeter creará los hilos justo antes de que comiencen a ejecutar solicitudes, lo que puede ahorrar recursos.
Specify Thread lifetime	Si está marcado, puedes establecer la duración de la prueba en segundos.
Duration (seconds)	Si "Specify Thread lifetime" está marcado, aquí defines la duración de la prueba.
Startup delay (seconds)	Retraso antes de que el grupo de hilos comience a ejecutar.

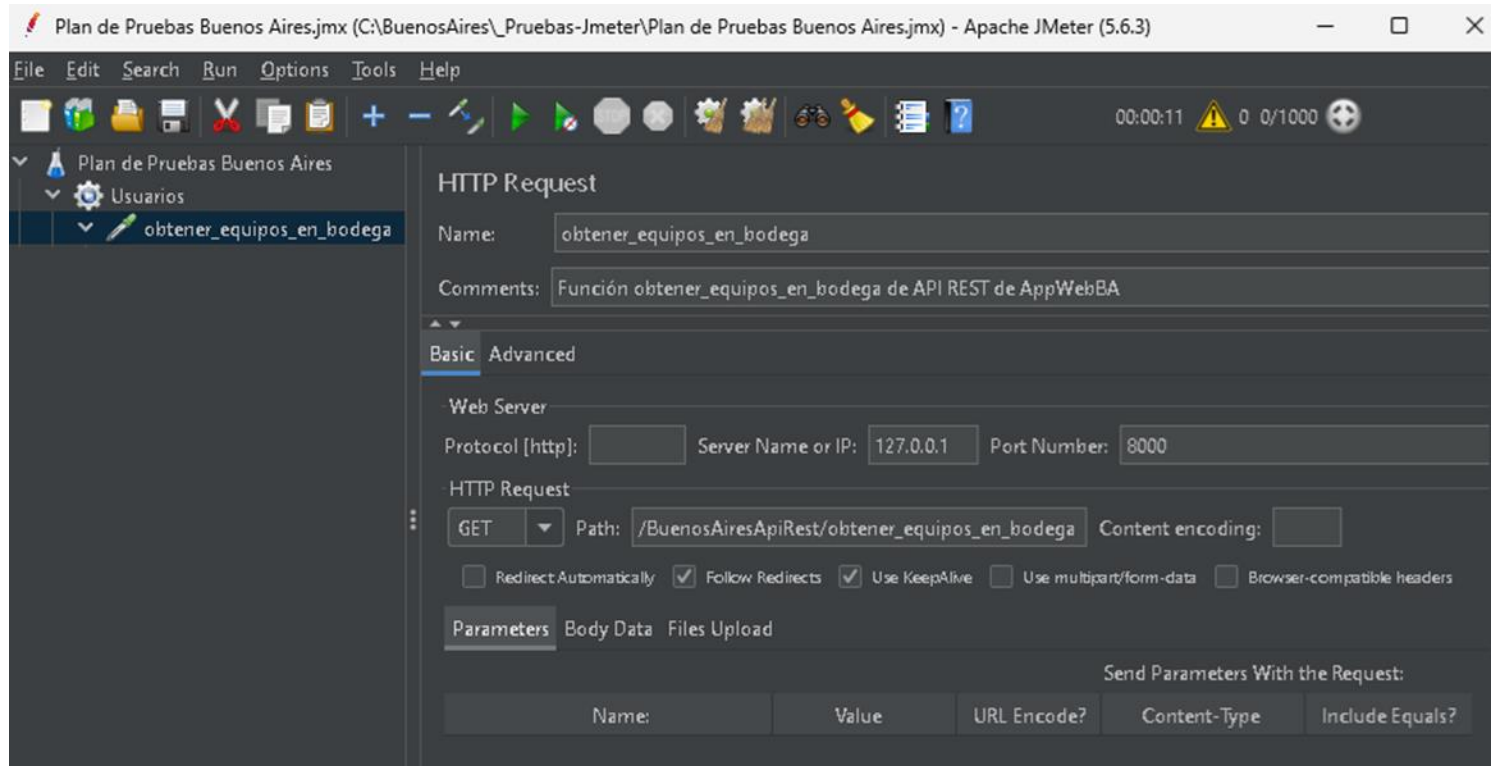


Agregar Peticiones o Solicitudes que Simulan las Acciones de los Usuarios (por ejemplo: Sampler tipo Http)

 obtener_equipos_en_bodega

En JMeter se puede configurar una solicitud HTTP, para simular la acción de un usuario, que solicita información por medio de una solicitud web utilizando alguno de los métodos del protocolo HTTP/HTTPS, por ejemplo: GET y POST. Con esta opción se puede, por ejemplo, probar la invocación a las funciones de una API REST. Para agregar un Sampler se debe hacer clic derecho sobre el Thread Group y seleccionar Add→Sampler→HTTP Request.

La ventana de configuración usando la pestaña "Basic" es la siguiente:



Name	Es el nombre que se le asigna a la petición HTTP, por ejemplo, "obtener_equipos_en_bodega". Sirve para identificar y organizar las diferentes peticiones dentro del plan de pruebas.
Comments	Permite agregar comentarios o descripciones sobre la petición HTTP. Por ejemplo: "Función obtener_equipos_en_bodega de API REST de AppWebBA", que indica que esta petición se utiliza para obtener información sobre los equipos en bodega a través de la API REST de la aplicación AppWebBA.
Web Server (Servidor Web):	
Protocol	Especifica el protocolo utilizado para la comunicación con el servidor web (HTTP o HTTPS). En este caso, se utiliza HTTP.
Server Name or IP	Nombre del servidor o IP. Indica la dirección IP o el nombre de dominio del servidor web al que se enviará la petición. En este caso, la dirección es "127.0.0.1", que es la dirección de localhost (el propio equipo).
Port Number	Número de puerto. Especifica el puerto en el que el servidor web está escuchando las peticiones. Por ejemplo: si el puerto es "8000", se trata del puerto por defecto de Django.
HTTP Request (Solicitud HTTP):	
Method	Método. Indica el método HTTP utilizado en la petición (GET, POST, PUT, DELETE, etc.). En este caso, se utiliza GET, que se utiliza para solicitar datos del servidor.
Path	Ruta. Especifica la ruta de acceso al recurso solicitado en el servidor web. En este caso, la ruta es "/BuenosAiresApiRest/obtener_equipos_en_bodega", lo que sugiere que se está accediendo a un punto final (end point) de la API REST llamado "obtener_equipos_en_bodega".
Content encoding	Codificación de contenido. Especifica la codificación de caracteres utilizada en la petición (por ejemplo, UTF-8).
Redirect Automatically	Redireccionar automáticamente. Si está marcada, JMeter seguirá automáticamente las redirecciones HTTP.
Follow Redirects	Seguir redirecciones. Similar a la opción anterior, pero permite configurar el número máximo de redirecciones a seguir.
Use KeepAlive	Usar KeepAlive. Si está marcada, JMeter mantendrá la conexión abierta con el servidor web después de cada petición, lo que puede mejorar el rendimiento.
Use multipart/form-data	Usar multipart/form-data. Si está marcada, JMeter enviará la petición utilizando el formato multipart/form-data, que se utiliza para enviar archivos junto con otros datos.
Browser-compatible headers	Encabezados compatibles con navegador. Si está marcada se enviará encabezados HTTP similares a los que envía un navegador web.

Parameters

La tabla de Parámetros permite agregar parámetros a una petición HTTP. Estos parámetros se pueden enviar de diferentes maneras, dependiendo del método HTTP que estés utilizando y la configuración de tu petición.

¿Cómo funciona?

Agregar parámetros: Puedes agregar parámetros a la tabla haciendo clic en el botón "Add" (Agregar). Cada parámetro tiene un nombre y un valor.

Codificación: JMeter codificará automáticamente los parámetros según la configuración de la petición. Por ejemplo, si estás utilizando el método GET, los parámetros se agregarán a la URL de la petición como una cadena de consulta (query string). Si estás utilizando el método POST, los parámetros se pueden enviar en el cuerpo de la petición, dependiendo de la configuración del encabezado `Content-Type`.

Envío: JMeter enviará los parámetros junto con la petición HTTP al servidor web.

¿Cuándo usar la tabla "Parameters"?

Método GET: Para enviar parámetros en la URL de la petición, por ejemplo:

`https://ejemplo.com/api?rut=12746940&nombre=Juan`

Para este ejemplo, en la tabla "Parameters", se debería agregar dos parámetros:

Nombre: rut, **Valor:** 12746940

Nombre: nombre, **Valor:** Juan

JMeter construiría automáticamente la URL completa y la enviaría al servidor.

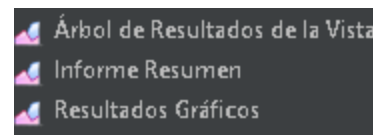
Método POST con Content-Type: application/x-www-form-urlencoded: Para enviar parámetros en el cuerpo de la petición en formato de formulario HTML.

Consideraciones:

Otros métodos y formatos: Si estás utilizando otros métodos HTTP (como POST, PUT, DELETE) o formatos de datos (como JSON), es posible que debas configurar la petición de manera diferente. Por ejemplo, para enviar datos JSON en el cuerpo de una petición POST, debes utilizar la sección "**Body Data**" en lugar de la tabla "**Parameters**".

Encabezados: Asegúrate de configurar los encabezados HTTP adecuados para indicar el formato de los datos que estás enviando (por ejemplo, **Content-Type: application/json** para datos JSON).

Agregar Recolectores de Datos o Analizadores de Resultados (Listeners)



Los Listeners en JMeter son elementos esenciales que te permiten visualizar y analizar los resultados de tus pruebas de rendimiento. Actúan como **"observadores" de las peticiones enviadas por JMeter** y **recopilan información sobre su rendimiento**, como tiempos de respuesta, códigos de estado, errores, etc.

En adelante, a los Listeners les llamaremos Recolectores de Datos.

¿Para qué sirven los recolectores de datos?

- **Visualizar resultados:** Los Listeners presentan los resultados de las pruebas en diferentes formatos, como tablas, gráficos, árboles y registros. Esto te permite ver fácilmente cómo se comportó tu aplicación bajo carga y detectar posibles problemas de rendimiento.
- **Analizar datos:** Los Listeners te proporcionan estadísticas detalladas sobre las peticiones, como el tiempo promedio de respuesta, la desviación estándar, el porcentaje de error y el rendimiento (throughput). Estos datos te ayudan a identificar cuellos de botella, errores y áreas de mejora en tu aplicación.
- **Depurar problemas:** Algunos Listeners, como el "View Results Tree" (Ver Árbol de Resultados), te permiten ver los detalles de cada petición y respuesta, lo que puede ser útil para depurar errores y comprender el comportamiento de la aplicación.
- **Generar informes:** Puedes configurar los Listeners para guardar los resultados en archivos (CSV, XML, etc.) o generar informes HTML. Esto te permite compartir los resultados con otros miembros del equipo o analizarlos más tarde con otras herramientas.

Tipos de recolectores de datos

JMeter ofrece una amplia variedad de recolectores de datos, cada uno con diferentes características y formas de presentar los resultados. Algunos de los recolectores de datos (Listeners) más comunes son:

1. **View Results Tree:** Muestra los detalles de cada petición y respuesta en formato de árbol. Útil para depurar errores.
2. **Summary Report:** Proporciona un resumen estadístico de todas las peticiones, incluyendo tiempos de respuesta, errores y rendimiento.
3. **Graph Results:** Muestra los tiempos de respuesta y el rendimiento en forma de gráficos.
4. **Aggregate Report:** Similar al Summary Report, pero agrupa los resultados por nombre de petición.
5. **Aggregate Graph:** Similar al Graph Results, pero agrupa los resultados por nombre de petición.
6. **Response Time Graph:** Muestra un gráfico de los tiempos de respuesta a lo largo del tiempo.
7. **Assertion Results:** Muestra los resultados de las aserciones (afirmaciones) que hayas configurado en tus peticiones.

Cómo usar los recolectores de datos:

- **Agregar Listeners:** Puedes agregar uno o más Listeners a tu plan de pruebas haciendo clic derecho en el elemento "Thread Group" (Grupo de Hilos) y seleccionando "Add" (Agregar) > "Listener" (Receptor).
- **Configurar Listeners:** Cada Listener tiene sus propias opciones de configuración, como el formato de salida, los datos a mostrar y el nombre del archivo de resultados.
- **Ejecutar la prueba:** Una vez que hayas configurado tus Listeners, ejecuta la prueba y observa los resultados en tiempo real o después de que la prueba haya finalizado.

Ejemplo de Análisis de Los Recolectores de Datos

Para agregar un recolector de datos, se debe hacer clic derecho sobre el Sampler, en nuestro ejemplo, sería sobre:

“obtener_equipos_en_bodega”

Luego agregaremos 3 recolectores de datos (Samplers):

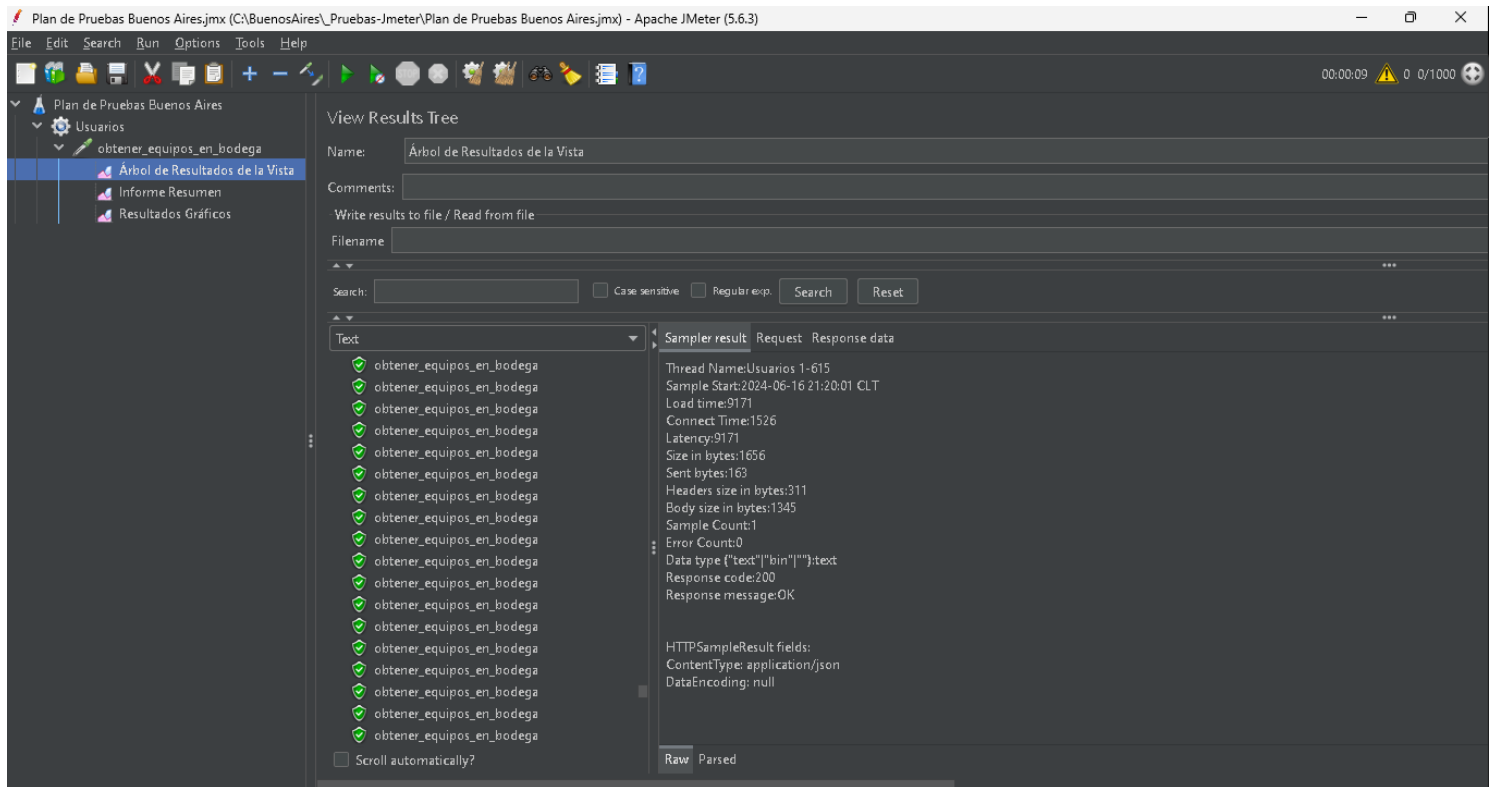
1. **View Results Tree:** Muestra los detalles de cada petición y respuesta en formato de árbol. Útil para depurar errores.
2. **Summary Report:** Proporciona un resumen estadístico de todas las peticiones, incluyendo tiempos de respuesta, errores y rendimiento.
3. **Graph Results:** Muestra los tiempos de respuesta y el rendimiento en forma de gráficos.

Antes de Iniciar (Start) las pruebas se debe levantar la API REST del proyecto Buenos Aires.

Los resultados de los 3 recolectores de datos son:

Recolector de Datos: Árbol de Resultados de la Vista (Sampler View Results Tree)

Este recolector de datos sirve para demostrar que se han realizado múltiples peticiones HTTP a la API REST con éxito. Sin embargo, se necesita más información para evaluar el rendimiento de la API y el propósito de la prueba.



Recolector de Datos: Informe de Resumen (Sampler View Results Tree)

Summary Report

Name: Informe Resumen

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
obtener_equipos_en_bodega	4100	5567	4	31566	7135.41	50.37%	23.5/min	6.00	0.04	15653.6
TOTAL	4100	5567	4	31566	7135.41	50.37%	23.5/min	6.00	0.04	15653.6

☐ Include group name in label? ☒ Save Table Header

Label	El nombre del sampler (solicitud HTTP, petición SOAP, etc.) que se está probando. En este caso, "obtener_equipos_en_bodega" es el nombre del sampler.
# Samples	El número total de muestras (solicitudes) enviadas durante la prueba para ese sampler en particular. Aquí, se enviaron 4100 solicitudes.
Average	El tiempo promedio (en milisegundos) que tardó cada muestra en completarse. En este caso, el tiempo promedio de respuesta fue de 5567 ms.
Min	El tiempo mínimo (en milisegundos) que tardó en completarse una muestra. Aquí, la solicitud más rápida tardó 4 ms.
Max	El tiempo máximo (en milisegundos) que tardó en completarse una muestra. En este caso, la solicitud más lenta tardó 31566 ms.
Std. Dev.	La desviación estándar de los tiempos de respuesta. Indica qué tan dispersos están los tiempos de respuesta alrededor del promedio. Una desviación estándar alta sugiere una mayor variabilidad en los tiempos de respuesta.
Error %	El porcentaje de muestras que resultaron en un error. Aquí, el 50.37% de las solicitudes fallaron.
Throughput	El rendimiento, es decir, el número de solicitudes por unidad de tiempo (en este caso, por minuto). En este ejemplo, se procesaron aproximadamente 23.5 solicitudes por minuto.
Received KB/sec	La cantidad de datos recibidos por segundo en kilobytes.
Sent KB/sec	La cantidad de datos enviados por segundo en kilobytes.
Avg. Bytes	El tamaño promedio (en bytes) de las respuestas recibidas.

¿Qué conclusiones se pueden obtener de estos datos y qué medidas se pueden tomar para solucionar los problemas que se presentan?

Problemas

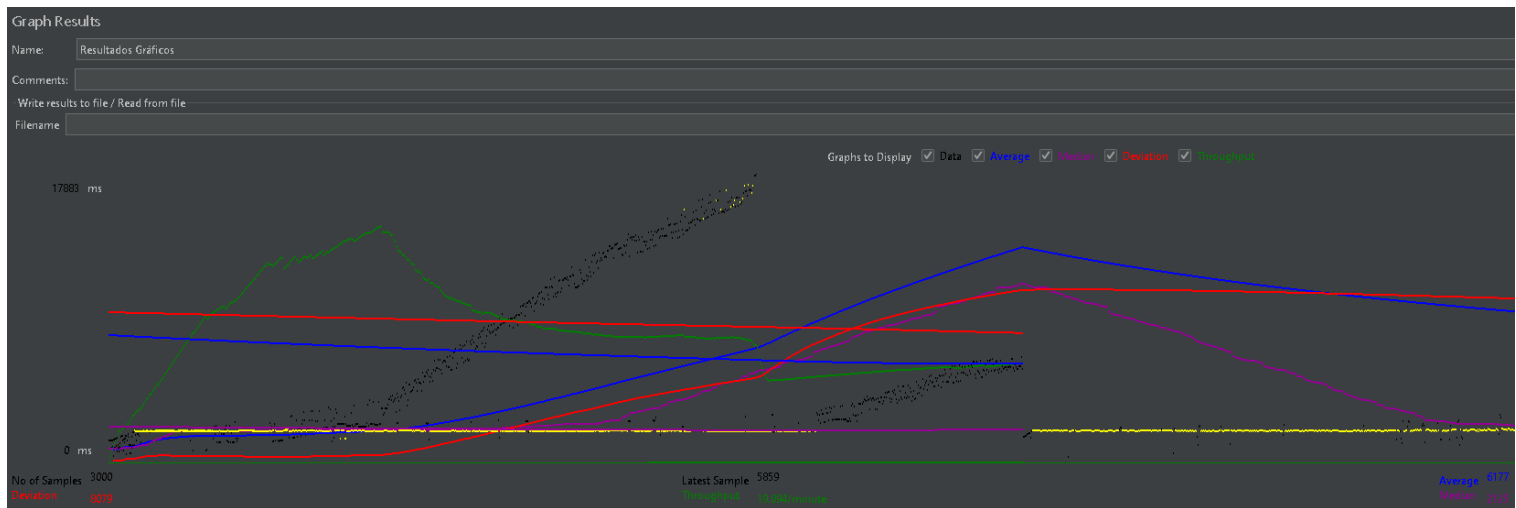
- Alto tiempo de respuesta promedio:** El tiempo promedio de respuesta de 5567 ms es bastante elevado, lo que sugiere que la operación de obtener los equipos en bodega es lenta. Esto podría deberse a varios factores, como consultas a la base de datos ineficientes, lógica de negocio compleja o falta de recursos en el servidor.
- Amplia variabilidad en los tiempos de respuesta:** La desviación estándar (Std. Dev.) de 7135.41 ms indica una gran variabilidad en los tiempos de respuesta. Esto significa que algunas solicitudes se procesan mucho más rápido que otras, lo que podría generar una experiencia de usuario inconsistente.
- Alto porcentaje de error:** Un valor de 50.37% es alarmante, ya que indica que la mitad de las solicitudes fallaron. Es crucial investigar la causa de estos errores, podrían deberse a problemas en el servidor, errores en la aplicación o limitaciones de recursos.
- Bajo rendimiento (throughput):** El rendimiento de 23.5 solicitudes por minuto es relativamente bajo, lo que sugiere que el sistema no puede manejar una gran cantidad de solicitudes simultáneas de manera eficiente.

Medidas para solucionar los problemas

- Optimizar consultas a la base de datos:** Analizar las consultas realizadas por la función "obtener_equipos_en_bodega" y optimizarlas para reducir el tiempo de ejecución. Se pueden utilizar índices, evitar consultas anidadas y optimizar el uso de joins.
- Mejorar la lógica de negocio:** Revisar el código de la función para identificar posibles cuellos de botella y optimizar la lógica de negocio para que sea más eficiente.
- Escalar recursos del servidor:** Si el servidor está sobrecargado, se puede considerar aumentar los recursos de hardware (CPU, memoria, disco) o escalar horizontalmente agregando más servidores.
- Investigar y corregir errores:** Analizar los registros de errores para identificar la causa de los errores en las solicitudes y corregirlos.
- Implementar caché:** Si los datos de los equipos en bodega no cambian con frecuencia, se puede implementar un mecanismo de caché para almacenar los resultados de las consultas y reducir la carga en el servidor.
- Considerar el uso de un balanceador de carga:** Si el problema es la capacidad del servidor para manejar múltiples solicitudes, se puede implementar un balanceador de carga para distribuir el tráfico entre varios servidores.

Recolector de Datos: Resultados Gráficos (Graph Results)

El recolector de datos de resultados gráficos muestra la siguiente información.



¿Qué problemas se pueden obtener de este gráfico y qué medidas se pueden tomar para solucionar los problemas que se presentan?

Se puede concluir lo siguiente:

Conclusiones

1. **Aumento del tiempo de respuesta:** El tiempo de respuesta promedio (línea azul) y la mediana (línea amarilla) aumentan a medida que avanza la prueba, lo que indica que el sistema se vuelve más lento bajo carga. Esto sugiere que el sistema no está escalando bien y podría tener dificultades para manejar un mayor número de usuarios o solicitudes.
2. **Alta variabilidad:** La desviación estándar (línea roja) también aumenta con el tiempo, lo que indica una gran variabilidad en los tiempos de respuesta. Esto significa que algunas solicitudes se procesan mucho más rápido que otras, lo que podría generar una experiencia de usuario inconsistente y frustrante.
3. **Aumento del rendimiento (throughput) inicial:** El rendimiento (línea verde) aumenta al principio de la prueba, pero luego se estabiliza y comienza a disminuir ligeramente. Esto sugiere que el sistema puede manejar una cierta carga inicial, pero tiene dificultades para mantener un alto rendimiento a medida que aumenta la carga.

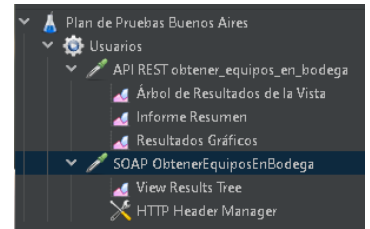
Medidas para solucionar los problemas

1. **Identificar cuellos de botella:** Analizar los resultados de la prueba y los registros del servidor para identificar los componentes del sistema que están causando el aumento del tiempo de respuesta y la disminución del rendimiento. Podría tratarse de consultas a bases de datos lentas, algoritmos ineficientes, falta de recursos o problemas de red.
2. **Optimizar el código y las consultas:** Optimizar el código de la aplicación y las consultas a la base de datos para mejorar la eficiencia y reducir el tiempo de procesamiento.
3. **Escalar los recursos:** Si el problema es la falta de recursos, aumentar la capacidad del servidor (CPU, memoria, disco) o agregar más servidores para distribuir la carga.
4. **Implementar caché:** Si los datos no cambian con frecuencia, implementar un mecanismo de caché para almacenar los resultados de las consultas y reducir la carga en el servidor.
5. **Utilizar un balanceador de carga:** Si el problema es la capacidad del servidor para manejar múltiples solicitudes, implementar un balanceador de carga para distribuir el tráfico entre varios servidores.
6. **Considerar un sistema de colas:** Si las solicitudes pueden ser procesadas de forma asíncrona, implementar un sistema de colas para gestionar las solicitudes y evitar que el servidor se sobrecargue.

Probando un Webservice SOAP de C# .NET

Como ejemplo se probará la función **ObtenerEquiposEnBodega** del **Webservice SOAP**, desarrollado en C#. NET en la **Capa de Servicio: BuenosAires.ServiceLayer** a la cual se puede acceder mediante la **URL: <http://localhost:54096/WsStockProducto.svc?wsdl>**

Con clic derecho en Usuarios→ Sampler→HTTP Request, se debe agregar una solicitud al método remoto **ObtenerEquiposEnBodega** llamada **“SOAP ObtenerEquiposEnBodega”**, que debe quedar configurado del siguiente modo:



HTTP Request

Name: SOAP ObtenerEquiposEnBodega

Comments: Función ObtenerEquiposEnBodega de Webservice SOAP de BuenosAires.ServiceLayer (http://localhost:54096/WsStockProducto.svc?wsdl)

Basic Advanced

Web Server

Protocol [http]: Server Name or IP: localhost Port Number: 54096

HTTP Request

POST Path: /WsStockProducto.svc Content encoding:

☐ Redirect Automatically ☒ Follow Redirects ☒ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers

Parameters Body Data Files Upload

```

1 <s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/">
2   <s:Body>
3     <ObtenerEquiposEnBodega xmlns="http://tempuri.org/" />
4   </s:Body>
5 </s:Envelope>

```

Server Name or IP	localhost
Port Number	54096
HTTP Request	POST
Path	/WsStockProducto.svc
Body Data	<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/"> <s:Body> <ObtenerEquiposEnBodega xmlns="http://tempuri.org/" /> </s:Body> </s:Envelope>

Con clic derecho en “SOAP ObtenerEquiposEnBodega” seleccionar Add→Config Element→HTTP Header Manager, y agregar las siguientes cabeceras:

HTTP Header Manager

Name: HTTP Header Manager

Comments:

Headers Stored in the Header Manager

Name:	Value
Content-Type	text/xml; charset=utf-8
SOAPAction	http://tempuri.org/IWsStockProducto/ObtenerEquiposEnBodega

Content-Type	text/xml; charset=utf-8
SOAPAction	http://tempuri.org/IWsStockProducto/ObtenerEquiposEnBodega